

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 8

дисциплина: *Архитектура компьютера*

Студент: Кабанов Иван Олегович

Группа: НКАбд-03-25

МОСКВА

2025 г.

Оглавление

Цель работы	3
8.3. Выполнение лабораторной работы	3
8.3.1. Реализация циклов в NASM	3
8.3.2. Обработка аргументов командной строки	6
8.4. Задание для самостоятельной работы	9
1.	9
Вывод	13

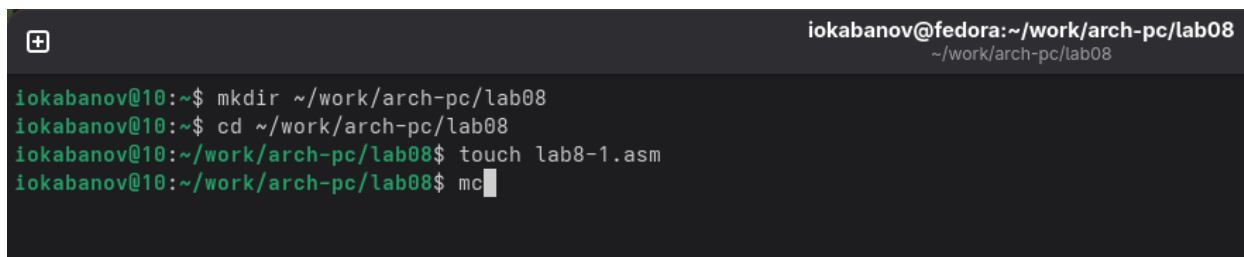
Цель работы

Приобретение навыков написания программ с использованием циклов и обработкой аргументов командной строки.

8.3. Выполнение лабораторной работы

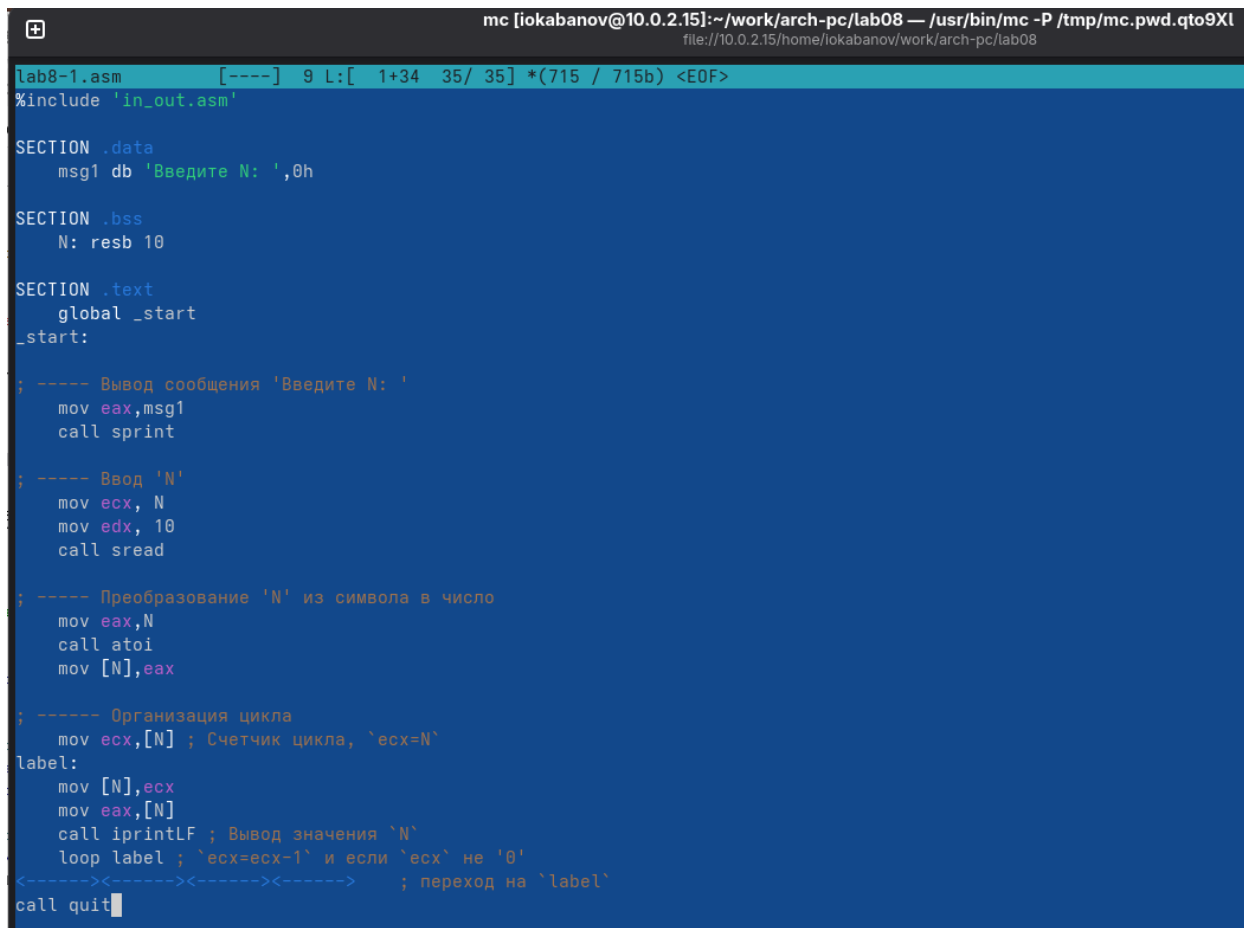
8.3.1. Реализация циклов в NASM

Создаем каталог для программ лабораторной работы № 8, переходим в него и создаем файл lab8-1.asm. Вводим в файл lab8-1.asm текст программы из листинга 8.1, создаем исполняемый файл и проверяем его работу.



```
iokabanov@fedora:~/work/arch-pc/lab08
iokabanov@10:~$ mkdir ~/work/arch-pc/lab08
iokabanov@10:~$ cd ~/work/arch-pc/lab08
iokabanov@10:~/work/arch-pc/lab08$ touch lab8-1.asm
iokabanov@10:~/work/arch-pc/lab08$ mc
```

Рис. 1. Создаем lab8-1.asm



```
mc [iokabanov@10.0.2.15]:~/work/arch-pc/lab08 — /usr/bin/mc -P /tmp/mc.pwd.qto9XI
file:///10.0.2.15/home/iokabanov/work/arch-pc/lab08
lab8-1.asm [----] 9 L:[ 1+34 35/ 35] *(715 / 715b) <E0F>
%include 'in_out.asm'

SECTION .data
    msg1 db 'Введите N: ',0h

SECTION .bss
    N: resb 10

SECTION .text
    global _start
    _start:

; ----- Вывод сообщения 'Введите N: '
    mov eax,msg1
    call sprint

; ----- Ввод 'N'
    mov ecx, N
    mov edx, 10
    call sread

; ----- Преобразование 'N' из символа в число
    mov eax,N
    call atoi
    mov [N],eax

; ----- Организация цикла
    mov ecx,[N] ; Счетчик цикла, 'ecx=N'
label:
    mov [N],ecx
    mov eax,[N]
    call iprintLF ; Вывод значения 'N'
    loop label ; 'ecx=ecx-1' и если 'ecx' не '0'
<-----><-----><-----> ; переход на 'label'
call quit
```

Рис. 2. Программа из листинга 8.1

```
iokabanov@fedora:~/work/arch-pc/lab08
~/work/arch-pc/lab08
iokabanov@10:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
iokabanov@10:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
iokabanov@10:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 12
12
11
10
9
8
7
6
5
4
3
2
1
iokabanov@10:~/work/arch-pc/lab08$
```

Рис. 3. Запуск программы из листинга 8.1

Видно, что использование регистра `ecx` в теле цикла `loop` привело к некорректной работе программы. Меняем текст программы, добавляя изменение значение регистра `ecx` в цикле. Создаем исполняемый файл и проверяем его работу. Так как в цикле `label` регистр `ecx` уменьшается на 2 из-за использования `sub` и `loop`, при $N=12$ он принимает значения 11, 9, 7, 5, 3, 1. Число проходов цикла не соответствует числу N , так как уменьшается на 2.

```
mc [iokabanov@10.0.2.15]:~/work/arch-pc/lab08 — /usr/bin/mc -P /tmp/mc.pwd.dBWfe2
file://10.0.2.15/home/iokabanov/work/arch-pc/lab08
lab8-1.asm  [-M--]  9 L: [ 1+35 36/ 36] *(743 / 743b) <E0F>
#include 'in_out.asm'

SECTION .data
    msg1 db 'Введите N: ',0h

SECTION .bss
    N: resb 10

SECTION .text
    global _start
_start:

; ----- Вывод сообщения 'Введите N: '
    mov eax,msg1
    call sprint

; ----- Ввод 'N'
    mov ecx, N
    mov edx, 10
    call sread

; ----- Преобразование 'N' из символа в число
    mov eax,N
    call atoi
    mov [N],eax

; ----- Организация цикла
    mov ecx,[N] ; Счетчик цикла, `ecx=N`
label:
    sub ecx,1 ; `ecx=ecx-1`
    mov [N],ecx
    mov eax,[N]
    call iprintLF ; Вывод значения `N`
    loop label ; `ecx=ecx-1` и если `ecx` не '0'
; -----><-----><-----> ; переход на `label`
call quit
```

Рис. 4. Отредактированная программа из листинга 8.1

```
iokabanov@fedora:~/work/arch-pc/lab08
~/work/arch-pc/lab08

iokabanov@10:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm -o lab8-1new.o
iokabanov@10:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1new lab8-1new.o
iokabanov@10:~/work/arch-pc/lab08$ ./lab8-1new
Введите N: 12
11
9
7
5
3
1
iokabanov@10:~/work/arch-pc/lab08$
```

Рис. 5. Проверка работы измененной программы из листинга 8.1

Для корректной работы программы вносим изменения в ее текст, добавляя команды `push` и `pop` для сохранения значения счетчика цикла `loop`. Создаем исполняемый файл и проверяем его работу. В данном случае, число проходов цикла соответствует значению `N`.

```
lab8-1.asm [-M--] 9 L: [ 1+38 39/ 39] *(885 / 885b) <EOF>
%include 'in_out.asm'

SECTION .data
    msg1 db 'Введите N: ',0h

SECTION .bss
    N: resb 10

SECTION .text
    global _start
_start:

; ----- Вывод сообщения 'Введите N: '
    mov eax,msg1
    call sprint

; ----- Ввод 'N'
    mov ecx,N
    mov edx,10
    call sread

; ----- Преобразование 'N' из символа в число
    mov eax,N
    call atoi
    mov [N],eax

; ----- Организация цикла
    mov ecx,[N] ; Счетчик цикла, 'ecx=N'
label:
    push ecx ; добавление значения ecx в стек
    sub ecx,1 ; 'ecx=ecx-1'
    mov [N],ecx
    mov eax,[N]
    call iprintLF ; Вывод значения 'N'
    pop ecx ; извлечение значения ecx из стека

    loop label ; 'ecx=ecx-1' и если 'ecx' не '0'
<-----><-----><-----><-----> ; переход на 'label'
call quit
```

Рис. 6. Программа из листинга 8.1 с использованием стека

```
iokabanov@fedora:~/work/arch-pc/lab08
~/work/arch-pc/lab08
iokabanov@10:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm -o lab8-1new.o
iokabanov@10:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1new lab8-1new.o
iokabanov@10:~/work/arch-pc/lab08$ ./lab8-1new
Введите N: 12
11
10
9
8
7
6
5
4
3
2
1
0
iokabanov@10:~/work/arch-pc/lab08$
```

Рис. 7. Корректная работа программы из листинга 8.1

8.3.2. Обработка аргументов командной строки

Создаем файл lab8-2.asm в каталоге ~/work/arch-pc/lab08 и вводим в него текст программы из листинга 8.2. Создаем исполняемый файл и запускаем его, указав аргументы: аргумент1, аргумент, 2, 'аргумент 3'. В данном случае, программой было обработано 4 аргумента.

```
iokabanov@fedora:~/work/arch-pc/lab08
~/work/arch-pc/lab08
iokabanov@10:~/work/arch-pc/lab08$ touch lab8-2.asm
iokabanov@10:~/work/arch-pc/lab08$ mc
```

Рис. 8. Создаем файл lab8-2.asm

```
mc [iokabanov@10.0.2.15]:~/work/arch-pc/lab08 — /usr/bin/mc -P /tmp/mc.pwd.FHiKlx
file://10.0.2.15/home/iokabanov/work/arch-pc/lab08

lab8-2.asm [-M-0] 50 L: [ 1+11 12/ 26] *(421 /1209b) 0010 0x00A
-----
; Обработка аргументов командной строки
-----

%include 'in_out.asm'

SECTION .text
global _start

_start:
    pop ecx ; Извлекаем из стека в `ecx` количество
<-----> ; аргументов (первое значение в стеке)
    pop edx ; Извлекаем из стека в `edx` имя программы
<-----> ; (второе значение в стеке)
    sub ecx, 1 ; Уменьшаем `ecx` на 1 (количество
<-----><----->; аргументов без названия программы)
next:
    cmp ecx, 0 ; проверяем, есть ли еще аргументы
    jz _end ; если аргументов нет выходим из цикла
<-----> ; (переход на метку `_end`)
    pop eax ; иначе извлекаем аргумент из стека
    call sprintf ; вызываем функцию печати
    loop next ; переход к обработке следующего
<-----><----->; аргумента (переход на метку `next`)
_end:
    call quit
```

Рис. 9. Программа из листинга 8.2

```
iokabanov@fedora:~/work/arch-pc/lab08
~/work/arch-pc/lab08

iokabanov@10:~/work/arch-pc/lab08$ nasm -f elf lab8-2.asm
iokabanov@10:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-2 lab8-2.o
iokabanov@10:~/work/arch-pc/lab08$ ./lab8-2 аргумент1 аргумент 2 'аргумент 3'
аргумент1
аргумент
2
аргумент 3
iokabanov@10:~/work/arch-pc/lab08$
```

Рис. 10. Запуск программы из листинга 8.2

Создаем файл lab8-3.asm в каталоге ~/work/archpc/lab08 и вводим в него текст программы из листинга 8.3. Проверяем корректность работы программы, запуская исполняемый файл и вводя аргументы: 12, 13, 7, 10, 5.

```
iokabanov@fedora:~/work/arch-pc/lab08
~/work/arch-pc/lab08

iokabanov@10:~/work/arch-pc/lab08$ touch lab8-3.asm
iokabanov@10:~/work/arch-pc/lab08$ mc
```

Рис. 11. Создаем файл lab8-3.asm

```
mc [iokabanov@10.0.2.15]:~/work/arch-pc/lab08 — /usr/bin/mc -P /tmp/mc.pwd.JSycdV
file://10.0.2.15/home/iokabanov/work/arch-pc/lab08

lab8-3.asm      [-M--] 36 L:[ 1+33 34/ 34] *(1532/1532b) <E0F>
#include 'in_out.asm'

SECTION .data
msg db "Результат: ",0

SECTION .text
global _start

_start:

    pop ecx ; Извлекаем из стека в `ecx` количество
<-----> ; аргументов (первое значение в стеке)
    pop edx ; Извлекаем из стека в `edx` имя программы
<-----> ; (второе значение в стеке)
    sub ecx,1 ; Уменьшаем `ecx` на 1 (количество
<-----> ; аргументов без названия программы)
    mov esi, 0 ; Используем `esi` для хранения
<-----> ; промежуточных сумм
next:
    cmp ecx,0h ; проверяем, есть ли еще аргументы
    jz _end ; если аргументов нет выходим из цикла
<-----> ; (переход на метку `_end`)
    pop eax ; иначе извлекаем следующий аргумент из стека
    call atoi ; преобразуем символ в число
    add esi,eax ; добавляем к промежуточной сумме
<-----> ; след. аргумент `esi=esi+eax`
    loop next ; переход к обработке следующего аргумента

_end:
    mov eax, msg ; вывод сообщения "Результат: "
    call sprint
    mov eax, esi ; записываем сумму в регистр `eax`
    call iprintLF ; печать результата
    call quit ; завершение программы
```

Рис. 12. Текст программы из листинга 8.3

```
iokabanov@fedora:~/work/arch-pc/lab08
~/work/arch-pc/lab08

iokabanov@10:~/work/arch-pc/lab08$ nasm -f elf lab8-3.asm
iokabanov@10:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-3 lab8-3.o
iokabanov@10:~/work/arch-pc/lab08$ ./lab8-3 12 13 7 10 5
Результат: 47
iokabanov@10:~/work/arch-pc/lab08$
```

Рис. 13. Запуск программы из листинга 8.3

Изменяем текст программы из листинга 8.3 для вычисления произведения аргументов командной строки. Заменяем `add` на `imul` и изменяем `mov esi,0` на `mov esi,1` для корректного выполнения программы. Создаем исполняемый файл и проверяем его работу.


```
mc [iokabanov@10.0.2.15]:~/work/arch-pc/lab08 — /usr/bin/mc -P /tmp/mc.pwd.IElzR2
file://10.0.2.15/home/iokabanov/work/arch-pc/lab08

lab8-3.asm      [-M--]  4 L: [ 1+33  34/ 34] *(1526/1526b) <EOF>
#include 'in_out.asm'

SECTION .data
msg db "Результат: ",0

SECTION .text
global _start

_start:

    pop ecx ; Извлекаем из стека в `ecx` количество
<-----> ; аргументов (первое значение в стеке)
    pop edx ; Извлекаем из стека в `edx` имя программы
<-----> ; (второе значение в стеке)
    sub ecx,1 ; Уменьшаем `ecx` на 1 (количество
<-----> ; аргументов без названия программы)
    mov esi, 1 ; Используем `esi` для хранения
<-----> ; промежуточных сумм. Для умножения начальное значение должно быть 1.
next:
    cmp ecx,0h ; проверяем, есть ли еще аргументы
    jz _end ; если аргументов нет выходим из цикла
<-----> ; (переход на метку `_end`)
    pop eax ; иначе извлекаем следующий аргумент из стека
    call atoi ; преобразуем символ в число
    imul esi,eax ; esi=esi*eax
    loop next ; переход к обработке следующего аргумента

_end:
    mov eax, msg ; вывод сообщения "Результат: "
    call sprint
    mov eax, esi ; записываем сумму в регистр `eax`
    call iprintLF ; печать результата
    call quit ; завершение программы
```

Рис. 14. Программа для вычисления произведения аргументов командной строки

```
iokabanov@fedora:~/work/arch-pc/lab08
~/work/arch-pc/lab08

iokabanov@10:~/work/arch-pc/lab08$ nasm -f elf lab8-3.asm -o lab8-3new.o
iokabanov@10:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-3new lab8-3new.o
iokabanov@10:~/work/arch-pc/lab08$ ./lab8-3new 8 7
Результат: 56
iokabanov@10:~/work/arch-pc/lab08$
```

Рис. 15. Запуск программы, вычисляющей произведение аргументов

8.4. Задание для самостоятельной работы

1.

Задание 1: Напишите программу, которая находит сумму значений функции $f(x)$ для $x = x_1, x_2, \dots, x_n$, т.е. программа должна выводить значение $f(x_1) + f(x_2) + \dots + f(x_n)$. Значения x_i передаются как аргументы. Вид функции $f(x)$ выбрать из таблицы 8.1 вариантов заданий в соответствии с вариантом,

полученным при выполнении лабораторной работы № 7. Создайте исполняемый файл и проверьте его работу на нескольких наборах $x = x_1, x_2, \dots, x_n$.

Так как в результате выполнения предыдущей лабораторной работы был получен вариант 16, пишем программу в файле task1.asm, которая находит сумму значений функции $f(x) = 30x - 11$, для $x = x_1, x_2, \dots, x_n$. Создаем исполняемый файл, запускаем его и проверяем его на нескольких наборах x . Убеждаемся в корректной работе программы.

Листинг task1.asm:

```
%include 'in_out.asm'
```

```
SECTION .data
```

```
msg db "Результат: ",0
```

```
SECTION .text
```

```
global _start
```

```
_start:
```

```
    pop ecx
```

```
    pop edx
```

```
    sub ecx, 1
```

```
    mov esi, 0
```

```
next:
```

```
    cmp ecx, 0h
```

```
    jz _end
```

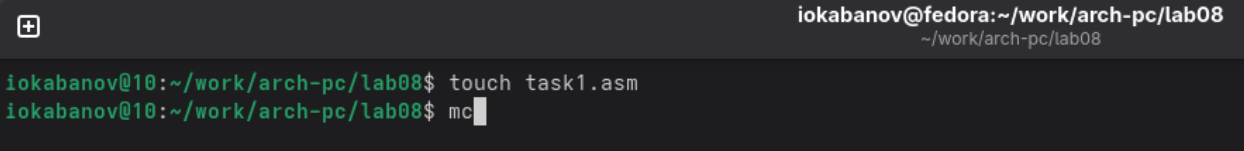
```
    pop eax
```

```
    call atoi
```

```
imul eax, 30 ; Умножаем x на 30. Теперь eax = 30*x
sub eax, 11 ; Вычитаем 11. Теперь eax = 30*x - 11
add esi, eax ; Добавляем результат функции f(x) к общей сумме
               ; esi = esi + (30x - 11)
loop next

_end:
mov eax, msg
call sprint
mov eax, esi ; Переносим накопленную сумму в eax для вывода
call iprintLF

call quit
```

A terminal window with a dark background. The title bar shows a window icon, a plus sign, and the text 'iokabanov@fedora:~/work/arch-pc/lab08'. The terminal content shows two lines of commands: 'touch task1.asm' and 'mc', both preceded by the prompt 'iokabanov@10:~/work/arch-pc/lab08\$'.

```
iokabanov@10:~/work/arch-pc/lab08$ touch task1.asm
iokabanov@10:~/work/arch-pc/lab08$ mc
```

Рис. 16. Создание файла task1.asm

```
mc [iokabanov@10.0.2.15]:~/work/arch-pc/lab08 — /usr/bin/mc -P /tmp/mc.pwd.ARaX5V
file://10.0.2.15/home/iokabanov/work/arch-pc/lab08

task1.asm [----] 0 L: 1+35 36/ 36] *(735 / 735b) <EOF>
#include 'in_out.asm'

SECTION .data
msg db "Результат: ",0

SECTION .text
global _start

_start:
    pop ecx.
    pop edx.
    sub ecx, 1.
    ....
    mov esi, 0.

next:
    cmp ecx, 0h.
    jz _end.....
    ....
    pop eax.....
    call atoi...
    ....
    imul eax, 30 ; Умножаем x на 30. Теперь eax = 30*x
    sub eax, 11 ; Вычитаем 11. Теперь eax = 30*x - 11
    add esi, eax ; Добавляем результат функции f(x) к общей сумме
                  ; esi = esi + (30x - 11)
    loop next....

_end:
    mov eax, msg.
    call sprint
    mov eax, esi ; Переносим накопленную сумму в eax для вывода
    call iprintLF
    ....
    call quit
```

Рис. 17. Текст программы task1.asm

```
iokabanov@fedora:~/work/arch-pc/lab08
~/work/arch-pc/lab08

iokabanov@10:~/work/arch-pc/lab08$ nasm -f elf task1.asm
iokabanov@10:~/work/arch-pc/lab08$ ld -m elf_i386 -o task1 task1.o
iokabanov@10:~/work/arch-pc/lab08$ ./task1 1 2 3
Результат: 147
iokabanov@10:~/work/arch-pc/lab08$ ./task1 8 1 2 2
Результат: 346
iokabanov@10:~/work/arch-pc/lab08$ ./task1 1 4 5 2 3
Результат: 395
iokabanov@10:~/work/arch-pc/lab08$
```

Рис. 18. Проверка работы программы task1 на нескольких наборах x

Вывод

В ходе выполнения лабораторной работы были приобретены навыки написания программ с использованием циклов и обработкой аргументов командной строки. Также были получены знания о структуре данных стек и особенностях работы с ней.

